

在 AI 時代建構應用程式

來源：<https://codelabs.developers.google.com/building-applications-in-the-ai-era?hl=zh-tw>

步驟數：9

在本研究室中，您將使用 Google 的生成式 AI 產品，在 Gemini Cloud Assist 的協助下，在 Google Cloud 中建構基礎架構。

目錄

1. 總覽
2. 設定和需求
3. 工作 0：檢查工作站叢集
4. 工作 1：啟用及使用 Gemini Cloud Assist
5. 工作 2：在 Vertex AI Agent Builder 中為烹飪建議聊天機器人建立搜尋應用程式
6. 工作 3：在 Colab Enterprise 筆記本中載入及清理資料，並使用 Gemini Code Assist 輔助
7. 工作 4：在 Vertex AI Agent Builder 中建立食譜生成器的搜尋應用程式
8. 工作 5：在 Gemini 的協助下，規劃 Python 和 Streamlit 網頁應用程式的核心架構
9. 工作 6：將網頁應用程式部署至 Cloud Run

1. 總覽

在本實驗室中，您將使用 Google 的生成式 AI 產品，在 Gemini Cloud Assist 的輔助下，於 Google Cloud 中建構基礎架構；使用 Data Canvas 的自然語言轉 SQL 功能查詢 BigQuery 資料；在 Gemini Code Assist 的輔助下，於 Colab Enterprise Jupyter 筆記本和 Eclipse Theia (Visual Studio Code) 中編寫程式碼；以及在 Vertex AI Agent Builder 中，整合以 Cloud Storage 和 BigQuery 基礎來源為基礎的 AI 搜尋和即時通訊功能。

我們的目標是建立名為「AI Recipe Haven」的食譜和烹飪網站。這個網站會以 Python 和 Streamlit 建構，並包含兩個主要頁面。「烹飪建議」會代管我們使用 Gemini 和 Vertex AI Agent Builder 建立的聊天機器人，並以一組食譜做為基礎來源，提供烹飪建議及回答烹飪相關問題。食譜搜尋功能是由 Gemini 驅動的搜尋引擎，這次的基礎是 BigQuery 食譜資料庫。

如果您在練習中遇到任何程式碼問題，所有程式碼檔案的解決方案都位於練習的 GitHub 存放區中，也就是 [solution](#) 分支。

目標

在本實驗室中，您將瞭解如何執行下列工作：

- 啟用及使用 Gemini Cloud Assist
 - 在 Vertex AI Agent Builder 中為烹飪建議聊天機器人建立搜尋應用程式
 - 在 Colab Enterprise 筆記本中載入及清理資料，並使用 Gemini Code Assist 輔助
 - 在 Vertex AI Agent Builder 中建立食譜生成器的搜尋應用程式
 - 在 Gemini 的協助下，規劃 Python 和 Streamlit 網頁應用程式的核心架構
 - 將網頁應用程式部署至 Cloud Run
 - 將「烹飪建議」頁面連結至我們的食譜搜尋 Agent Builder 應用程式
 - (選用) 將食譜搜尋頁面連結至食譜搜尋 Agent Builder 應用程式
 - (選用) 探索最終應用程式
-

步驟 2 · 約 0 分鐘

2. 設定和需求

實驗室活動啟動須知

請詳閱以下操作說明。實驗室活動會計時，中途無法暫停。點選「Start Lab」後就會開始計時，顯示可使用 Google Cloud 資源的時間。

您會在實際雲端環境中完成 Qwiklabs 實作實驗室活動，而非模擬或示範環境。為此，我們會提供新的暫時憑證，供您在實驗室活動期間登入及存取 Google Cloud。

需求條件

為了順利完成這個實驗室，請先確認：

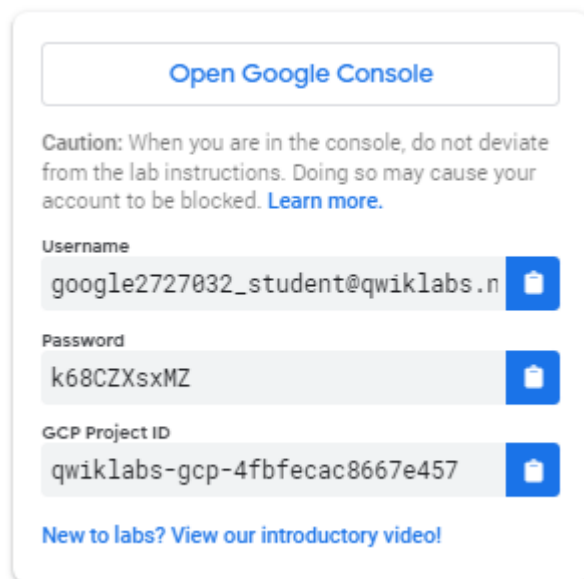
- 可以使用標準的網際網路瀏覽器 (Chrome 瀏覽器為佳)。
- 已預留充足時間可完成實驗室。

注意：如果您擁有個人 Google Cloud 或專案，請勿用於本實驗室。

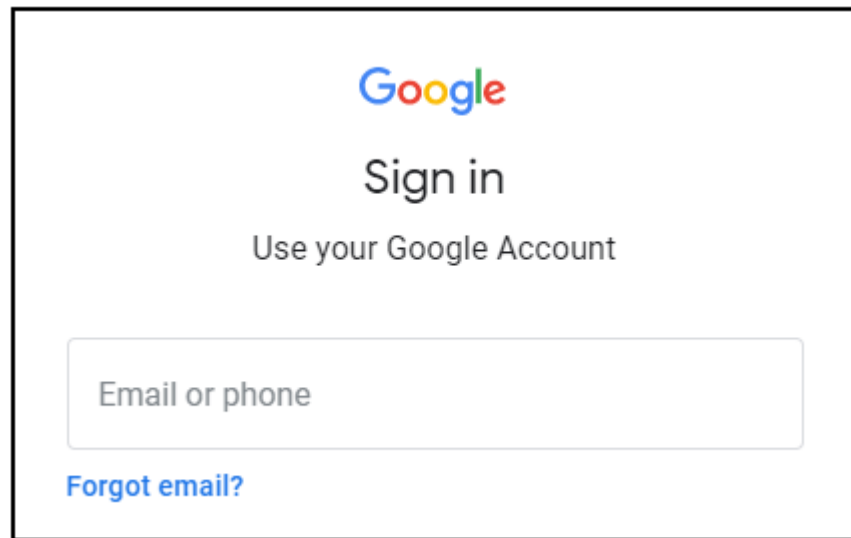
注意：如果您使用的是 Pixelbook，請在無痕視窗執行本實驗室。

如何開始研究室及登入 Google Cloud 控制台

1. 按一下「Start Lab」按鈕。如果實驗室會產生費用，畫面中會出現選擇付款方式的彈出式視窗。左側面板中會顯示在此研究室中您必須使用的暫時憑證。

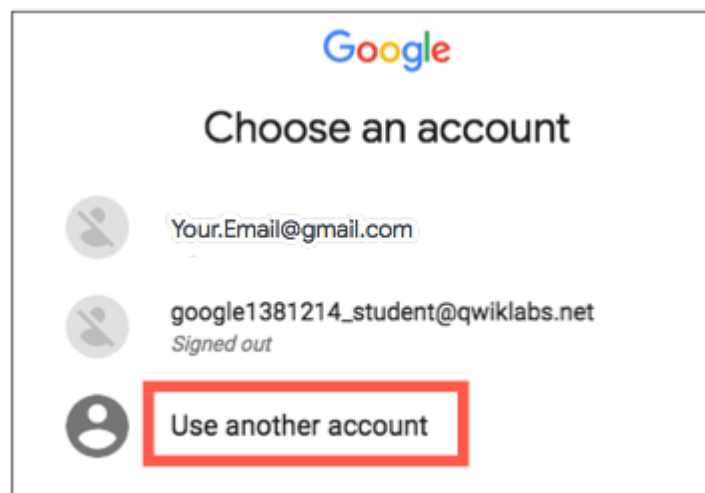


2. 複製使用者名稱，然後點選「Open Google Console」。接著，實驗室會啟動相關資源，並開啟另一個分頁，顯示「登入」頁面。



提示：請在不同視窗中並排開啟分頁。

如果看到「選擇帳戶」頁面，請按一下「使用其他帳戶」。



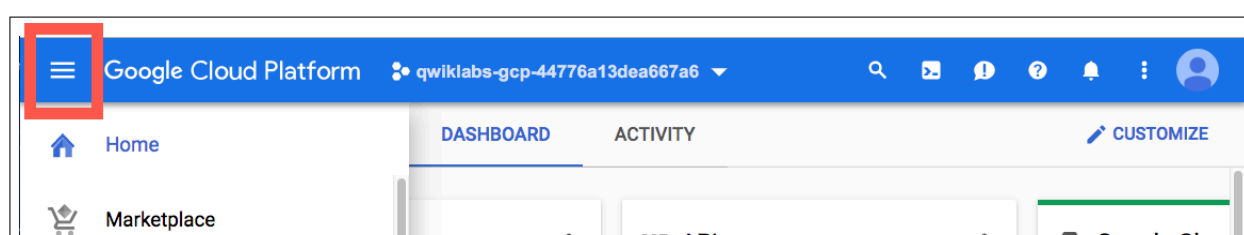
3. 在「登入」頁面，貼上您在「連線詳細資訊」面板中複製的使用者名稱，然後複製並貼上密碼。

重要事項：請務必使用「連線詳細資訊」面板中的憑證，而非您自己的 Qwiklabs 憑證。請勿在這個實驗室中使用您自己的 Google Cloud 帳戶，否則會產生費用。

1. 接受條款及細則。
2. 由於這是臨時帳戶，請勿新增救援選項或雙重驗證機制。
3. 請勿申請免費試用。

Cloud 控制台稍後會在這個分頁中開啟。

注意：點選左上方的「導覽選單」，即可查看 Google Cloud 產品與服務清單。



步驟 3 · 約 0 分鐘

3. 工作 0：檢查工作站叢集

在本實驗室的後續部分，您將使用 Google Cloud Workstation 執行一些開發工作。本實驗室的啟動程序應已開始建立工作站叢集。請先確認叢集正在建構中，再繼續操作。

1. 在 Google Cloud 控制台，使用搜尋方塊前往「Cloud Workstations」。
 2. 使用左側導覽選單查看「叢集管理」。
 3. 如果叢集正在更新，表示一切正常，可以繼續進行工作 1。如果沒有看到任何狀態的叢集，請重新整理頁面。如果還是沒有看到「正在更新 (建構)」叢集，請使用這些操作說明左上方的按鈕**結束實驗室**，然後重新啟動實驗室。
-

4. 工作 1：啟用及使用 Gemini Cloud Assist

在這項工作中，我們將啟用並使用 Gemini Cloud Assist。在 Google Cloud 控制台中工作時，Gemini Cloud Assist 可以提供建議、協助您建構、設定及監控 Google Cloud 基礎架構，甚至建議 `gcloud` 指令及編寫 Terraform 指令碼。

1. 如要啟用 Cloud Assist，請點選 Cloud 控制台 UI 頂端的「搜尋」方塊，然後選取「詢問 Gemini」（或「詢問 Gemini 關於 Cloud 控制台」）。
2. 捲動至頁面的「必要 API」部分，然後**啟用** Gemini for Google Cloud API。
3. 如果沒有立即看到即時通訊介面，請按一下「開始即時通訊」。首先，請 Gemini 說明使用 Cloud Workstations 的一些優點。請花幾分鐘時間查看生成的回覆。
4. 接著，請詢問 Agent Builder 的優點，以及如何協助生成回覆。
5. 最後，我們來看看比較結果。在 Google Cloud 控制台的 Gemini 對話視窗中，提出下列問題：

```
What are the major steps to creating a search app grounded in a  
GCS data source using Vertex AI Agent builder?
```

6. 現在，在非無痕視窗中前往公開的 Gemini 網站，視需要登入並提出相同問題。回應是否相同或至少相似？具體步驟為何？兩者是否明顯有差異？無論如何，請記住這些回應，以便進行後續步驟。

注意：如果您嘗試使用臨時 Qwiklabs 帳戶執行上述步驟，系統會封鎖您的帳戶。如果貴機構不允許使用 Gemini 網頁應用程式，導致您的公司帳戶也遭到封鎖，請直接略過這個步驟。這不會影響您完成這項練習。

5. 工作 2：在 Vertex AI Agent Builder 中為烹飪建議聊天機器人建立搜尋應用程式

我們即將建構的網站會提供烹飪建議頁面，其中包含聊天機器人，可協助使用者尋找烹飪相關問題的解答。這項功能會以 Gemini 為基礎，並參考 70 本公版食譜。Gemini 回答問題時，會以食譜為準。

1. 使用 Cloud 控制台搜尋方塊前往 **Vertex AI**。在資訊主頁中，點按「啟用所有建議的 API」。如果系統彈出視窗，指出需要啟用 Vertex AI API，請一併啟用。
2. 使用搜尋功能前往 **Agent Builder**，然後點選「繼續並啟用這個 API」。
3. 如 Gemini 在先前的建議中指出，在 Agent Builder 中建立搜尋應用程式的第一步，是建立權威資料來源。使用者搜尋時，Gemini 會瞭解問題和智慧回覆的撰寫方式，但會從基礎來源尋找回覆中使用的資訊，而不是從內建知識庫提取。

在左側選單中，前往「資料儲存庫」並建立資料儲存庫。4. 我們目前使用的公共領域食譜位於外部專案的 Cloud Storage bucket 中，選取 Cloud Storage 來源類型。5. 檢查與要匯入資訊類型相關的預設選項，但不要變更。將匯入類型設為「資料夾」，並使用 `labs.roittraining.com/labs/old-cookbooks` 做為 bucket 路徑，然後按一下「繼續」。6. 將資料儲存庫命名為 `old-cookbooks`。編輯並將 ID 變更為 `old-cookbooks-id`，然後**建立**資料儲存庫。

Vertex AI Agent Builder 支援多種應用程式類型，而資料儲存庫是各類型的真實來源。搜尋應用程式適合一般用途和搜尋。Chat 應用程式適用於 Dataflow 驅動的聊天機器人/語音機器人應用程式中的生成流程。建議應用程式有助於建立更完善的建議引擎。而 Agent 應用程式則是用於建立生成式 AI 驅動的代理程式。最終，Agent 可能最能滿足我們的需求，但目前產品仍處於預覽階段，因此我們將繼續使用 Search 應用程式類型。7. 使用左側選單前往「應用程式」，然後按一下「建立應用程式」。8. 選取「搜尋」應用程式類型。檢查各種選項，但不要變更。為應用程式命名：`cookbook-search`。編輯並將應用程式 ID 設為 `cookbook-search-id`。將公司設為 `Google`，然後按一下「繼續」。9. 勾選您在幾個步驟前建立的 `old-cookbooks` 資料儲存庫，然後**建立**搜尋應用程式。

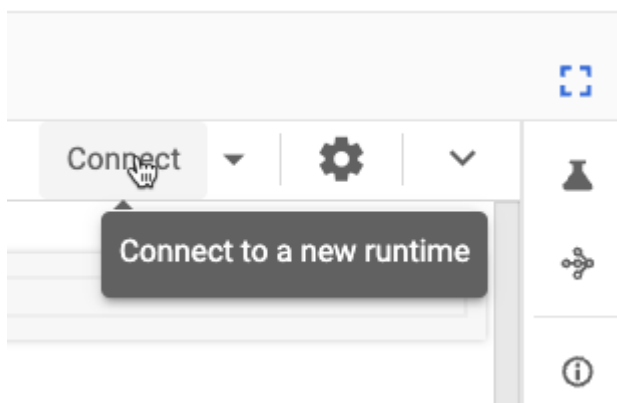
查看「活動」分頁時，您可能會發現系統仍在匯入食譜並建立索引。Agent Builder 需要 5 分鐘以上的時間，才能為我們提供的 70 本食譜中的數千個頁面建立索引。在等待期間，我們來載入並清除食譜資料庫資料，以供食譜產生器使用。

6. 工作 3：在 Colab Enterprise 筆記本中載入及清理資料，並使用 Gemini Code Assist 輔助

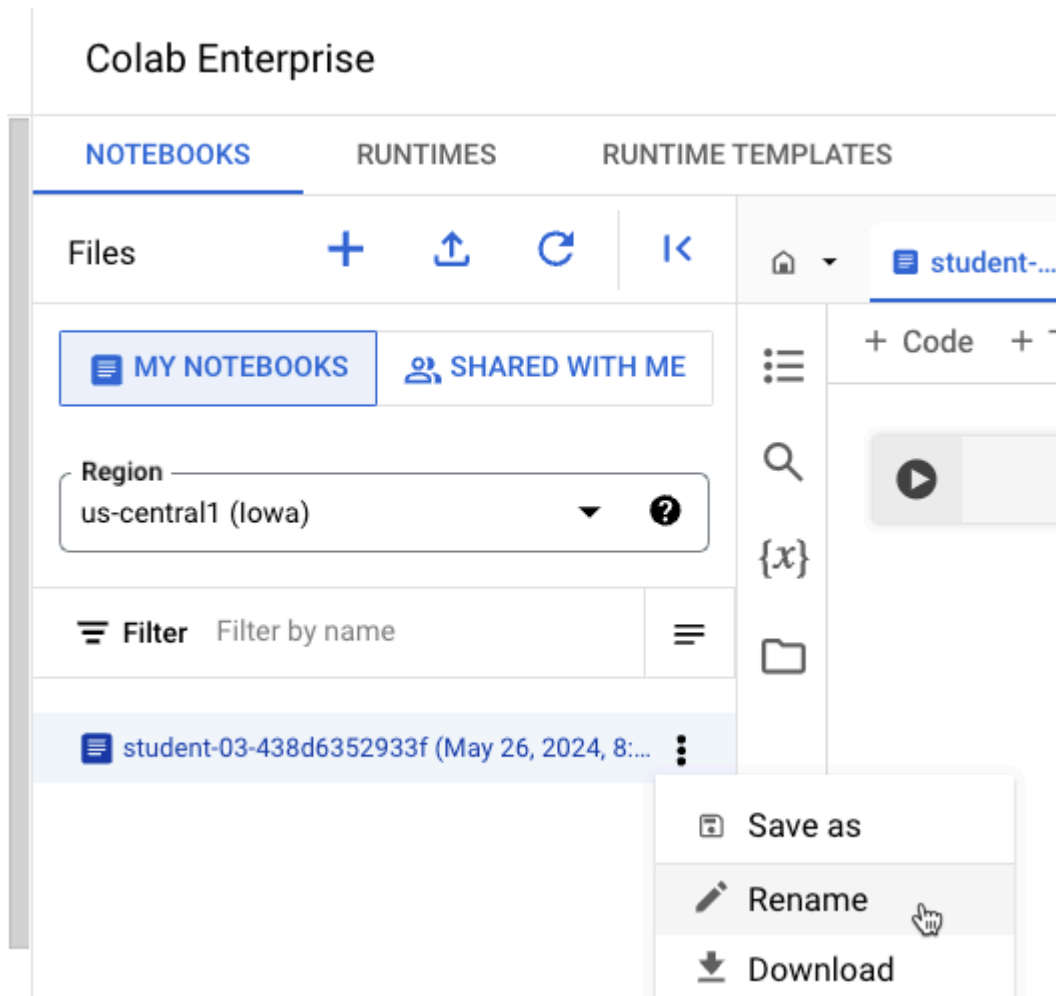
Google Cloud 提供幾種主要方式，讓您使用 Jupiter 筆記本。我們將使用 Google 的最新產品 Colab Enterprise。部分使用者可能熟悉 Google 的 [Colab](#) 產品，個人和機構通常會使用這項產品，在免費環境中試用 Jupiter 筆記本。Colab Enterprise 是 Google Cloud 的商業產品，與其他 Google 雲端產品完全整合，並充分運用 GCP 環境的安全防護和法規遵循功能。

Colab Enterprise 提供多項功能，其中一項是整合 Google 的 Gemini Code Assist。您可以在多種程式碼編輯器中使用 Code Assist，在編寫程式碼時取得建議和 [內嵌](#) 建議。我們會在處理食譜資料時運用這項生成式助理。

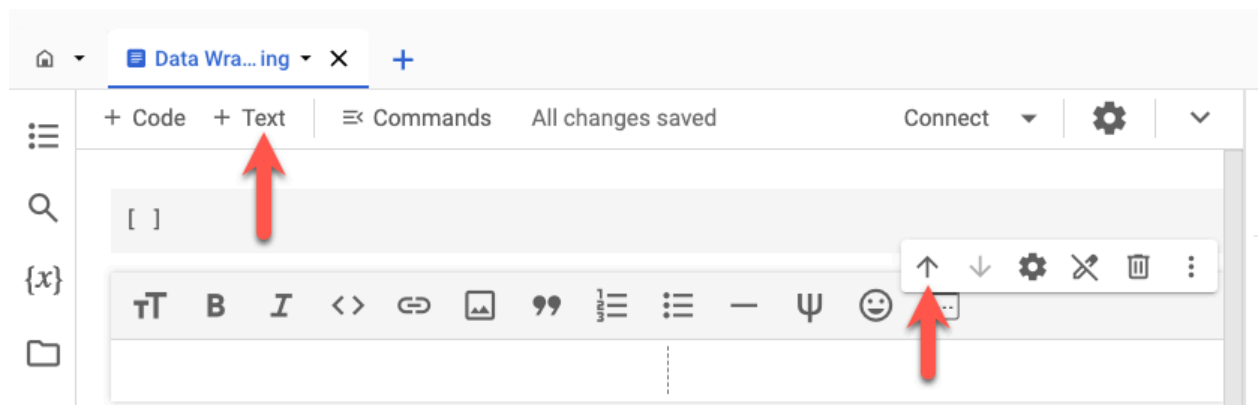
1. 使用搜尋功能前往 **Colab Enterprise**，然後**建立筆記本**。如果系統顯示邀請你試用新版 Colab 功能的訊息，請關閉該訊息。如要啟動筆記本的運算執行階段，請按下新筆記本右上角的「連線」。



2. 在 Colab Enterprise 的「檔案」窗格中，點選目前筆記本名稱旁的三點圖示選單，即可重新命名 `Data Wrangling`。



3. 建立新的「+ 文字」方塊，然後使用向上箭頭將其移至頁面上的第一個儲存格。



4. 編輯文字方塊並輸入：

```
# Data Wrangling

Import the Pandas library
```

5. 在剛建立的文字區塊下方的程式碼區塊中，開始輸入 `imp`，Gemini Code Assist 應會以灰色建議其餘匯入內容。按下 **Tab** 鍵即可接受建議。

```
import pandas as pd
```

6. 在匯入程式碼方塊下方，建立另一個文字方塊並輸入：

```
Create a Pandas DataFrame from: gs://labs.roitraining.com/labs/recipes/recipe_dataset.csv.  
View the first few records.
```

7. 建立及編輯其他程式碼區塊。再次輸入 `df`，然後檢查 Gemini Code Assistant 生成的程式碼。如果系統在生成的建議上方顯示 Python 關鍵字的自動完成下拉式清單，請按下 `Esc` 鍵，查看淺灰色的建議程式碼。再次按下 `Tab` 鍵即可接受建議。如果建議未包含 `head()` 函式呼叫，請新增。

```
df = pd.read_csv('gs://labs.roitraining.com/labs/recipes/recipe_dataset.csv')  
df.head()
```

8. 按一下第一個程式碼儲存格 (您在其中匯入 Pandas)，然後使用「Commands」選單或鍵盤執行所選儲存格。在鍵盤上按下 **Shift + Enter** 鍵，即可執行儲存格並將焦點移至下一個儲存格 (視需要建立儲存格)。請等待儲存格執行完畢再繼續操作。

注意：如果儲存格尚未執行，左側會顯示 `[]`。儲存格執行時，你會看到轉動的工作動畫。儲存格完成後，會顯示數字，例如 `[13]`。9. 執行將 CSV 載入 DataFrame 的儲存格。等待檔案載入，並檢查前五列資料。這是我們要載入 BigQuery 的食譜資料，最終會用來做為食譜產生器的基礎。10. 建立新的程式碼區塊，然後輸入下列註解。輸入註解後，移至下一行程式碼，系統應會顯示建議 `df.columns`。接受後執行儲存格。

```
# List the current DataFrame column names
```

我們剛才示範了在 Jupyter 筆記本中，你可以透過兩種方式向 Gemini Code Assist 尋求協助：程式碼儲存格上方的文字儲存格，或是程式碼儲存格內的註解。Jupyter 筆記本中的程式碼儲存格註解效果良好，但這個方法也適用於任何支援 Google Gemini Code Assist 的其他 IDE。

11. 我們來稍微清理一下資料欄。將資料欄 `Unnamed: 0` 重新命名為 `id`，並將 `link` 重新命名為 `uri`。使用您選擇的提示 > 程式碼技術建立程式碼，然後在滿意時執行儲存格。

```
# Rename the column 'Unnamed: 0' to 'id' and 'link' to 'uri'  
df.rename(columns={'Unnamed: 0': 'id', 'link': 'uri'}, inplace=True)
```

12. 移除 `source` 和 `NER` 欄，然後使用 `head()` 查看前幾列。再次請 Gemini 幫忙。執行最後兩行程式碼，並檢查結果。

```
# Remove the source and NER columns  
df.drop(columns=['source', 'NER'], inplace=True)  
df.head()
```

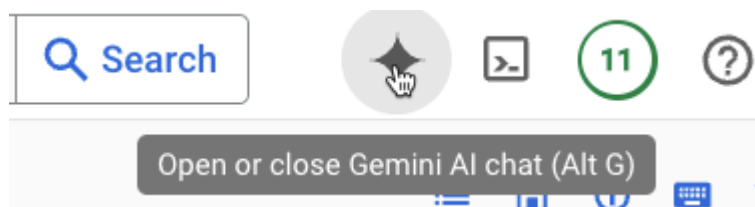
13. 讓我們看看資料集中有多少筆記錄。同樣地，請先選擇提示技巧，看看 Gemini 是否能協助您生成程式碼。

```
# Count the records in the DataFrame  
df.shape # count() will also work
```

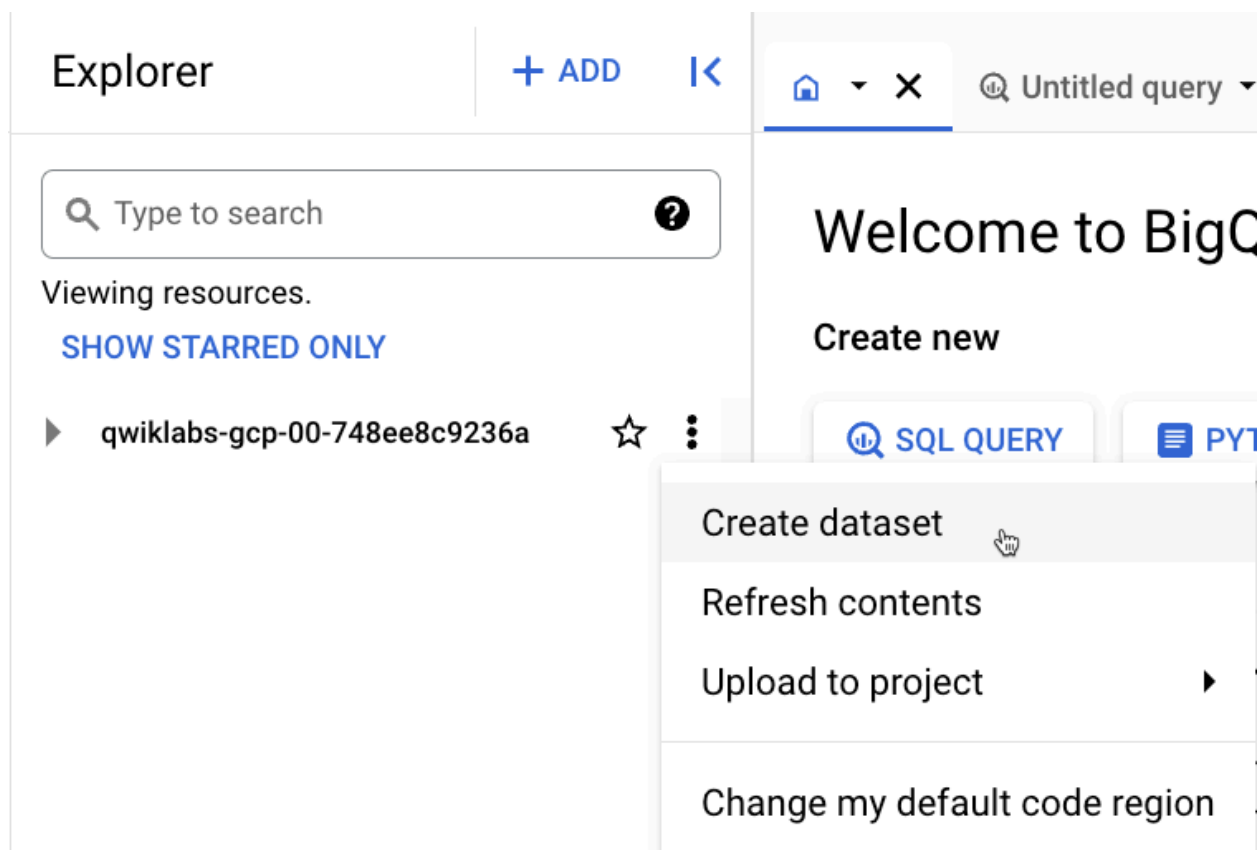
14. 223 萬筆記錄可能比我們有時間做的食譜還多。在 Agent Builder 中建立索引的過程可能需要很長時間，因此不適合今天的練習。為求折衷，我們將取樣 15 萬份食譜，並以此為基礎進行作業。使用「提示」>「程式碼」方法，擷取範例並儲存在名為 `dfs` (`s` 代表 `small`) 的新 DataFrame 中。

```
# Sample out 150,000 records into a DataFrame named dfs
dfs = df.sample(n=150000)
```

- 食譜來源資料已準備好載入 BigQuery。載入資料前，請先前往 BigQuery，準備好用來存放資料表的資料集。在 Google Cloud 控制台使用搜尋方塊，前往 **BigQuery**。您可以按一下滑鼠右鍵開啟 **BigQuery**，然後在新瀏覽器分頁中開啟。
- 如果 Gemini AI Chat 面板尚未顯示，請使用 Cloud Shell 右上角的 Gemini 標誌開啟。如果系統要求您再次啟用 API，請按下啟用鍵或重新整理頁面。執行提示：What is a dataset used for in BigQuery? 瀏覽回覆內容後，請提出以下要求：How can I create a dataset named recipe_data using the Cloud Console? 比較結果與下列幾個步驟。



- 在 BigQuery 的「Explorer」窗格中，點選專案 ID 旁邊的三點「查看動作」選單。然後選取「建立資料集」。



- 提供資料集和 `recipe_data` 的 ID。將位置類型保留為「美國」，然後「建立資料集」。如果收到資料集已存在的錯誤訊息，請直接繼續。

在 BigQuery 中建立資料集後，請切換回筆記本並插入資料。19. 切換回 Colab Enterprise 中的資料整理筆記本。在新的程式碼儲存格中，建立名為 `project_id` 的變數，並用來保留目前的專案 ID。在操作說明的左上方，也就是「End Lab」按鈕下方，即可找到目前的專案 ID。您也可以 Cloud 控制台首頁找到這項資訊。將值指派給 `project_id` 變數，然後執行儲存格。

```
# Create a variable to hold the current project_id
project_id='YOUR_PROJECT_ID'
```

20. 使用提示 > 程式碼方法建立程式碼區塊，將 DataFrame `dfs` 插入剛才建立的資料集 `recipe_data` 中，並命名為 `recipes`。執行儲存格。

```
dfs.to_gbq(destination_table='recipe_data.recipes', project_id=project_id,
if_exists='replace')
```

7. 工作 4：在 Vertex AI Agent Builder 中建立食譜生成器的搜尋應用程式

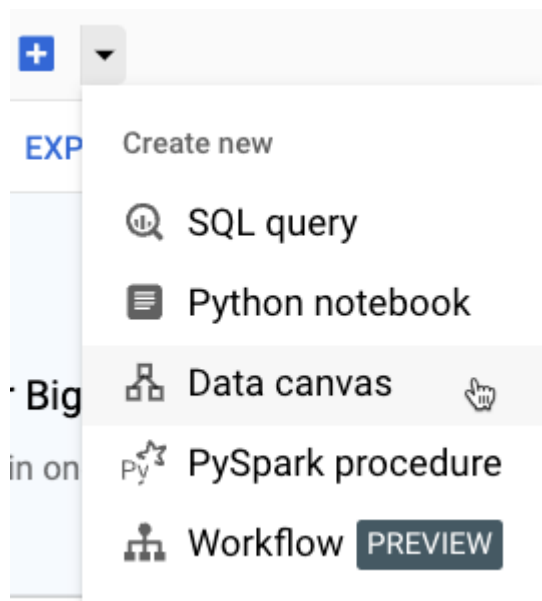
太棒了，我們已建立食譜資料表，現在要使用這個資料表，為食譜產生器建構基礎資料來源。我們將採用與烹飪聊天機器人類似的做法。我們將使用 Vertex AI Agent Builder 建立資料儲存庫，然後將該儲存庫做為搜尋應用程式的真實來源。

如要請 Gemini 在 Google Cloud 控制台中提醒您建立 Agent Builder 搜尋應用程式的步驟，或按照下列步驟操作，請隨意。

1. 使用搜尋功能前往 **Agent Builder**。開啟「資料儲存庫」並「建立資料儲存庫」。這次請**選取** BigQuery 資料儲存庫類型。
2. 在表格選取儲存格中，按下「瀏覽」並搜尋 `recipes`。選取表格旁的圓形按鈕。如果看到其他 `qwiklabs-gcp-...` 專案的配方，請務必**選取**屬於您的專案。

注意：如果點選 `recipes` 而不是選取旁邊的圓形按鈕，瀏覽器會開啟新分頁，並將您帶往 BigQuery 的表格總覽頁面。只要關閉瀏覽器分頁，然後在 Agent Builder 中選取圓形按鈕，3. 檢查其餘預設選項，但不要變更，然後點選「繼續」。4. 在結構定義審查頁面中，檢查初始預設設定，但請勿變更任何項目。**繼續**。5. 將資料儲存庫命名為 `recipe-data`。編輯資料儲存庫 ID，並設為 `recipe-data-id`。**建立**資料儲存庫。6. 使用左側導覽選單前往「應用程式」，然後「建立應用程式」。7. 再次**選取**「搜尋」應用程式。將應用程式命名為 `recipe-search`，並將 ID 設為 `recipe-search-id`。將公司名稱設為 `Google`，然後按一下「繼續」。8. 這次請檢查 **recipe-data** 資料來源。**建立**應用程式。

資料庫表格需要一段時間才能建立索引。在等待期間，我們來試試 BigQuery 的新功能「資料畫布」，看看能否找到一兩道有趣的食譜。9. 使用搜尋框前往 **BigQuery**。在 BigQuery Studio 頂端，按一下最右側分頁標籤旁的向下箭頭，然後選取「資料畫布」。將區域設為 `us-central1`。



10. 在資料畫布搜尋方塊中搜尋 `recipes`，然後將資料表新增至畫布。
11. 食譜資料表的視覺化呈現方式會載入 BigQuery 資料畫布。您可以探索資料表的結構定義、預覽資料表中的資料，以及檢查其他詳細資料。在表格表示法下方，按一下「查詢」。
12. 畫布會載入或多或少典型的 BigQuery 查詢對話方塊，但多了一個項目：查詢視窗上方有一個文字方塊，您可以在其中提示 Gemini 提供協助。讓我們看看能否在範例中找到一些蛋糕食譜。執行下列提示 (輸入文字並按下 Enter/Return 鍵，觸發 SQL 生成)：

Please select the title and ingredients for all the recipes with a title that contains the word cake.

13. 查看生成的 SQL 查詢。確認無誤後，**執行查詢**。

14. 還不錯吧！您可以先試試其他提示和查詢，再繼續下一個步驟。進行實驗時，請嘗試使用較籠統的提示，瞭解哪些提示有效，哪些無效。舉例來說，以下提示：

```
Do I have any chili recipes?
```

(別忘了執行新查詢) 傳回辣椒食譜清單，但未列出食材，直到我修改為：

```
Do I have any chili recipes? Please include their title and ingredients.
```

(沒錯，我在提示時會說「請」。我的媽媽一定會以我為榮。)

我發現其中一份辣椒食譜含有蘑菇，誰會在辣椒裡加蘑菇？我請 Gemini 協助排除這些食譜。

```
Do I have any chili recipes? Please include their title and ingredients, and ignore any recipes with mushrooms as an ingredient.
```

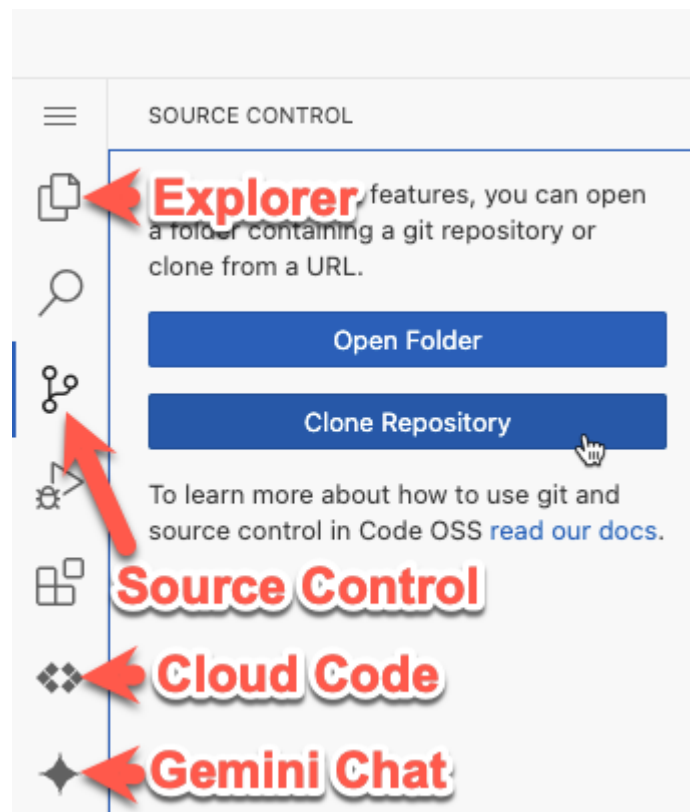
8. 工作 5：在 Gemini 的協助下，規劃 Python 和 Streamlit 網頁應用程式的核心架構

兩個 Vertex AI Agent Builder 資料儲存庫都已建立索引，搜尋應用程式也差不多準備就緒，接下來就來建構網頁應用程式。

我們將在工作時運用 Gemini Code Assist。如要進一步瞭解如何在 Visual Studio Code 中使用 Gemini Code Assist，請參閱[這份文件](#)。

我們將在 Google Cloud Workstation 中進行開發。這是一個雲端開發環境，在本例中，我們預先載入了 Eclipse Theia (開放原始碼的 Visual Studio Code)。本練習中的自動化指令碼已為我們建立 Cloud Workstations 叢集和設定，但我們仍需建立 Cloud Workstations 本身。如要進一步瞭解 Cloud Workstations 及其用途，請向 Gemini Cloud Assist 提問 :-)

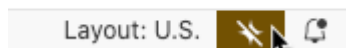
1. 使用搜尋功能前往 **Cloud Workstations**，然後「建立工作站」。將工作站命名為 `dev-env`，並使用 `my-config` 設定。**建立**工作站。
2. 稍候片刻，您就會在「我的工作站」清單中看到新的工作站。**啟動** `dev-env`，然後**啟動**開發環境。
3. 工作站編輯器會在新的瀏覽器分頁中開啟，過一會兒，您應該會看到熟悉的 Theia (Visual Studio Code) 介面。在介面左側展開「Source Control」分頁，然後按下「Clone Repository」。



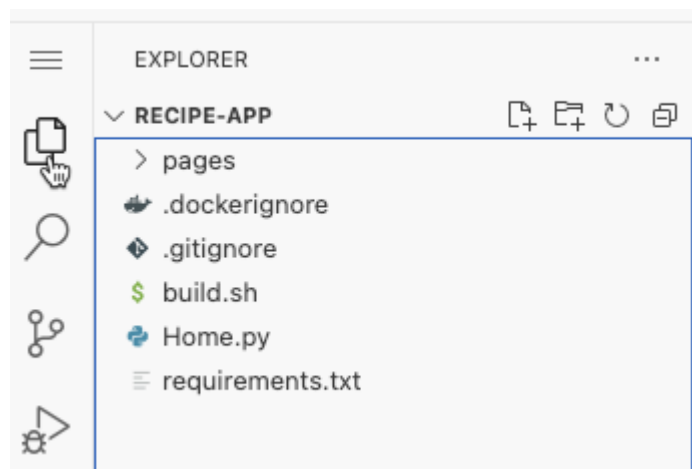
4. 在存放區網址中輸入 `https://github.com/haggman/recipe-app`。將存放區複製到 `user` 資料夾，然後**開啟**複製的存放區進行編輯。
5. 在瀏覽複製的資料夾並開始處理網頁應用程式之前，我們需要讓編輯器的 Cloud Code 外掛程式登入 Google Cloud，並啟用 Gemini。現在就來瞭解吧！在編輯器的左下方，按一下「Cloud Code - Sign in」。如果沒有看到連結，請稍候片刻再檢查一次。



6. 終端機視窗會顯示一長串網址。在瀏覽器中開啟網址，然後按照步驟授予 Cloud Code 存取 Google Cloud 環境的權限。請務必使用練習臨時帳戶 `student-...` 進行驗證，而非個人 Google Cloud 帳戶。在最後一個對話方塊中複製驗證碼，然後貼回 Cloud Workstation 瀏覽器分頁中等待的終端機視窗。
7. 過一會兒，編輯器左下方的 Cloud Code 連結會變更為「Cloud Code - No Project」。按一下新連結即可選取專案。編輯器頂端應會開啟指令調色盤。按一下「Select a Google Cloud project」，然後選取「qwiklabs-gcp-...」專案。稍待片刻，編輯器左下方的連結就會更新，顯示專案 ID。這表示 Cloud Code 已成功附加至工作專案。
8. Cloud Code 連線至專案後，即可啟用 Gemini Code Assist。在編輯器介面的右下角，按一下劃掉的 Gemini 標誌。編輯器左側會開啟 Gemini Chat 窗格。按一下「選取 Google Cloud 專案」。指令調色盤開啟後，請選取 **qwiklabs-gcp-...** 專案。如果正確按照步驟操作 (且 Google 沒有任何異動)，現在應該會看到已啟用的 Gemini 對話視窗。



9. 最後，我們來設定編輯器終端機視窗。使用漢堡選單 > 「View」 > 「Terminal」開啟終端機視窗。執行 `gcloud init`。再次使用連結，允許 Cloud Shell 終端機對 `qwiklabs-gcp-...` 專案執行作業。系統詢問時，請選取 `qwiklabs-gcp-...` 專案的數字選項。
10. 太棒了！終端機、Gemini Chat 和 Cloud Code 設定都已完成，請開啟「Explorer」分頁，花幾分鐘瀏覽目前專案中的檔案。



11. 在檔案總管中開啟 `requirements.txt` 檔案進行編輯。切換至 **Gemini** 對話窗格，然後提出以下問題：

From the dependencies specified in the requirements.txt file, what type of application are we building?

12. 因此，我們將使用 Python 和 Streamlit 建構互動式網路應用程式，與 Vertex AI 和 Discovery Engine 互動。目前先著重於網頁應用程式元件。如 Gemini 所述，[Streamlit](#) 是一個架構，可使用 Python 建構以資料為基礎的網頁應用程式。現在請詢問：

Does the current project's folder structure seem appropriate for a Streamlit app?

這也是 Gemini 容易出錯的地方。Gemini 可以存取您目前在編輯器中開啟的檔案，但無法查看整個專案。問題範例：

Given the below, does the current project's file and folder structure seem appropriate for a Streamlit app?

```
- build.sh
- Home.py
- requirements.txt
- pages
-- Cooking_Advice.py
-- Recipe_Search.py
```

取得更合適的答案？

13. 請提供更多有關 Streamlit 的資訊：

What can you tell me about Streamlit?

很好，我們可以看到 Gemini 提供了一份不錯的摘要，包括優點和缺點。

14. 如要瞭解缺點，可以問：

What are the major downsides or shortcomings?

請注意，我們不必說「of Streamlit」，因為 Gemini Chat 是對話式 (多輪)。由於我們正在進行即時通訊，Gemini 知道我們談論的內容。如要清除 Gemini 程式碼對話記錄，請使用 Gemini 程式碼對話視窗頂端的垃圾桶圖示。

9. 工作 6：將網頁應用程式部署至 Cloud Run

太棒了，我們已建立核心應用程式結構，但這些結構是否都能正常運作？更重要的是，我們應該在 Google Cloud 中代管哪些內容？

1. 在 Gemini 對話視窗中提問：

```
If I containerize this application, what compute technologies  
in Google Cloud would be best for hosting it?
```

2. 請注意，如果您尚未在 IDE 中工作，也可以使用 Google Cloud Assist。開啟 Google Cloud 控制台，然後開啟 Gemini Cloud Assist 並提出以下問題：

```
If I have a containerized web application, where would be the  
best place to run it in Google Cloud?
```

這兩組建議是否相同？你是否同意/不同意任何建議？請注意，Gemini 是生成式 AI 助理，就像真人助理一樣，你可能不會完全同意 Gemini 的說法。不過，在 Google Cloud 和程式碼編輯器中工作時，如果身邊有這麼一位幫手，效率會大幅提升。

3. 如果是無狀態的短期容器化網頁應用程式，Cloud Run 會是不錯的選擇。在程式碼編輯器的 Gemini 對話視窗中，嘗試輸入下列提示：

```
What steps would be required to run this application in  
Cloud Run?
```

4. 看來我們首先需要建立 Dockerfile。使用編輯器在專案資料夾的根目錄中建立名為 `Dockerfile` 的檔案。請務必不要誤放在頁面資料夾中。開啟檔案進行編輯。
5. 我們將使用側邊 Gemini 對話面板建立 `Dockerfile`。請使用類似下方的提示。當結果顯示在對話視窗中時，請使用建議 Dockerfile 上方複製圖示旁的 +，將建議的程式碼插入 Dockerfile。

```
Create a Dockerfile for the application in the current folder.  
The dependencies are defined in requirements.txt and I want you  
to use the Python 3 slim bookworm base image.
```

Gemini 不一定會對相同提示詞傳回相同回覆。我第一次要求 Gemini 提供 Dockerfile 時，得到的檔案與我建議您使用的完全相同。我剛收到以下建議：

```

```docker
Base image
FROM python:3-bookworm-slim

Set working directory
WORKDIR /app

Install dependencies
RUN apt-get update && apt-get install -y \
 build-essential \
 libpq-dev \
 gcc \
 python3-dev \
 && rm -rf /var/lib/apt/lists/*

Install pip and virtualenv
RUN pip install --upgrade pip virtualenv

Create virtual environment
RUN python3 -m venv venv

Activate virtual environment
WORKDIR /app/venv/bin
RUN . activate

Install Streamlit and libraries from requirements.txt
RUN pip install -r requirements.txt

Copy application files
COPY . /app

Expose port 8501 for Streamlit
EXPOSE 8501

Start Streamlit app
CMD ["streamlit", "run", "main.py"]

```

That's a heck of a Dockerfile. I'd simplify it a bit. We don't need the apt-get section as anything needed for Python is already in our base image. Also, using a virtual environment in a Python container is a waste of space, so I'd remove that. The expose command isn't strictly necessary, but it's fine. Also, it's trying to start main.py which I don't have.

6. Tweak the Dockerfile so it resembles the following:

```
FROM python:3.11-slim-bookworm
```

```
WORKDIR /app
```

COPY requirements.txt . RUN pip install --no-cache-dir --upgrade pip &&  
pip install --no-cache-dir -r requirements.txt

COPY . ◦

CMD ["streamlit", "run", "Home.py"]

7. We need a place in Google cloud where we can store our container image. Let's get a little help from Gemini. In the Google Cloud Console Gemini Cloud Assist ask:

在 Google Cloud 中，儲存 Docker 映像檔的最佳位置為何？

8. If one of the answers you received is the Google Container Registry, then I guess Gemini hasn't gotten word that GCR is deprecated. Again, just like human assistants, you may get out of date or simply wrong answers (hallucinations). Always make sure to consider your choices carefully, even when Gemini is recommending something.

Let's go with Artifact Registry. Ask Gemini Cloud Assist how to create a docker registry in Artifact Registry named cooking-images.

如何使用 gcloud 在 Artifact Registry 中建立 Docker 登錄檔？

9. Now ask Gemini how you could use Cloud Build to build a new image named `recipe-web-app` from the Dockerfile in the current folder.

如何使用 gcloud，從我們剛建立的 Artifact Registry 存放區中，以同名映像檔建構名為 recipe-web-app 的新 Cloud Run 服務？

10. To save you a little time, I've created a script that will create the Artifact Registry repo (if needed), use Cloud Build to build and push the image to the repo, and finally to deploy the application to Cloud Run. In your code editor use the **Explorer** view to open `build.sh` and explore the file.

11. Gemini can operate via the chat window, but it can also work directly in your code file using comments, like we used in the Data Wrangling notebook, and it also may be invoked using Control+i on Windows or Command+i on Mac. Click somewhere in the build.sh script file, activate Gemini using the appropriate Command+i / Control+i command.



12. At the prompt enter the below. Examine and **Accept** the change.

請為目前的檔案加上註解。

How cool is that?! How many times have you had to work with someone else's code, only to have to waste time gaining a base understanding of their commentless work before you can even start making your changes. Gemini to the rescue!

13. Let's build and deploy our application. In the terminal window execute the ``build.sh`` file.

`. build.sh`

14. If you watch the build process, first it will build the Artifact Registry docker repo. Then, it uses Cloud Build to create the container image from the Dockerfile in the local folder (since we didn't supply a ``cloudbuild.yaml``). Lastly, the docker image will be deployed into a new Cloud Run service. At the end of the script you'll get a Cloud Run test URL to use.

Open the returned link in a new tab of your browser. Take a moment and explore the application's structure and pages. Nice, now we need a hook in our generative AI functionality.

`## Task 7: Connect the Cooking Advice page to our cookbook-search Agent Builder app`

We have the framework for the web application running, but we need to connect the two work pages to our two Vertex AI Agent Builder search apps. Let's start with Cooking Advice.

1. In the Google Cloud console use search to navigate to **Chat** in Vertex AI.
2. In the right hand settings page pane set the model to **gemini-1.5-flash-002**. Slide the output token limit up to the max so the model can return longer answers if needed. Open the **Safety Filter Settings**. Set Hate speech, Sexually explicit content, and Harassment content to **Block some**. Set Dangerous content to **Block few** and **Save**. We're setting Dangerous Content a bit lower because talking about knives and cutting can be misinterpreted by Gemini as violence.
3. Slide on the toggle to enable **Grounding** then click **Customize**. Set the grounding source to **Vertex AI search** and for the datastore path use the following. Change `YOUR_PROJECT_ID` to the project ID found up near the End Lab button in these instructions, then **Save** the grounding settings

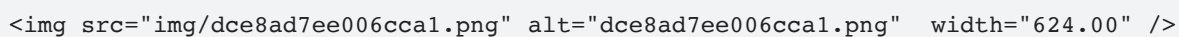
`projects/YOUR_PROJECT_ID/locations/global/collections/default_collection/dataStores/old-cookbooks-id`

**\*\*Note:\*\*** If you get an error then you either didn't change the project ID to your actual project ID, or you may have missed the step where you changed the old-cookbooks Agent Builder Data Store ID. Check your Agent Builder > Data Stores > old-cookbooks for its actual Data store ID.

4. Test a couple of chat messages. Perhaps start with the below. Try a few others if you like.

如何判斷番茄是否成熟？

5. The model works, now let's experiment with the code. Click **\*\*Clear Conversation\*\*** so our conversations don't become part of the code then click **\*\*Get Code\*\***.

 ``

6. At the top of the code window, press Open Notebook so we can experiment and perfect the code in Colab Enterprise before integrating it into our app.

7. Take a few minutes to familiarize yourself with the code. Let's make a couple of changes to adapt it to what we want. Before we start, run the first code cell to connect to the compute and install the AI Platform SDK. After the block runs you will be prompted to restart the session. Go ahead and do that.

8. Move to the code we pulled out of Vertex AI Studio. Change the name of the method `*multiturn_generate_content*` to ``start_chat_session``.

9. Scroll to the ``model = GenerativeModel(`` method call. The existing code defines the ``generation_config`` and ``safety_settings`` but doesn't actually use them. Modify the creation of the ``GenerativeModel`` so it resembles:

```
model = GenerativeModel("gemini-1.5-flash-002", tools=tools, generation_config=generation_config,
safety_settings=safety_settings,)
```

10. Lastly, add a final line to the method, just below ``chat = model.start_chat()``, so the function returns the ``chat`` object. The finished function should look like the below.

**\*\*Note:\*\*** DO NOT COPY this code into your notebook. It is simply here as a sanity check.

```
def start_chat_session(): vertexai.init(project="qwiklabs-gcp-02-9a7298ceaaec", location="us-central1") tools = [
Tool.from_retrieval(retrieval=grounding.Retrieval(source=grounding.VertexAISearch(datastore="projects/qwiklabs-gcp-
02-9a7298ceaaec/locations/global/collections/default_collection/dataStores/old-cookbooks-id"),)),] model =
GenerativeModel("gemini-1.5-flash-002", tools=tools, generation_config=generation_config,
safety_settings=safety_settings,) chat = model.start_chat() return chat
```



11. Scroll to the bottom of the code cell and change the final line calling the old function so it calls the new function name and stores the returned object in a variable `chat`. Once you are satisfied with your changes, run the cell.

```
chat = start_chat_session()
```

12. Create a new code cell and add the comment `# Use chat to invoke Gemini and print out the response`. Move to the next line and type resp and Gemini should auto complete the block for you. Update the prompt to `How can I tell if a tomato is ripe?`. Run the cell

```
response = chat.send_message("How can I tell if a tomato is ripe?") print(response)
```

13. That's the response alright, but the part we really want is that nested text field. Modify the codeblock to print just that section, like:

```
response = chat.send_message("How can I tell if a tomato is ripe?") print(response.candidates[0].content.parts[0].text)
```

14. Good, now that we have working chat code, let's integrate it into our web application. Copy all the contents of the code cell that creates the `start\_chat\_session` function (we won't need the test cell). If you click into the cell you can click the triple dot menu in the upper right corner and copy from there

```

```

15. Switch to your Cloud Workstation editor and open pages\Cooking\_Advice.py for editing.  
16. Locate the comment:

在這則訊息下方新增從筆記本複製的程式碼

17. Paste your copied code just below the above comment. Nice, now we have the section which drives the chat engine via a grounded call to Gemini. Now let's integrate it into Streamlit.  
18. Locate section of commented code directly below the comment:

以下是設定工作階段變數的程式碼

按照指示取消註解這個區塊

19. Uncomment this section of code (Up till the next ``Setup done, let's build the page UI`` section) and explore it. It creates or retrieves the chat and history session variables.

20. Next, we need to integrate the history and chat functionality into the UI. Scroll in the code until you locate the below comment.

以下是建立即時通訊介面的程式碼

按照指示取消註解下列程式碼

21. Uncomment the rest of the code below the comment and take a moment to explore it. If you like, highlight it and get Gemini to explain its functionality.

22. Excellent, now let's build the application and deploy it. When the URL comes back, launch the application and give the Cooking Advisor page a try. Perhaps ask it about ripe tomatoes, or the bot knows a good way to prepare brussels sprouts.

`. build.sh`

How cool is that! Your own personal AI cooking advisor :-)

## ## Task 8: (Optional) Connect the Recipe Search page to the recipe-search Agent Builder app

When we connected the Cooking Advice page to its grounded source, we did so using the Gemini API directly. For Recipe Search, let's connect to the Vertex AI Agent Builder search app directly.

1. In your Cloud Workstation editor, open the ``pages/Recipe_Search.py`` page for editing. Investigate the structure of the page.
2. Towards the top of the file, set your project ID.
3. Examine the ``search_sample`` function. This code more or less comes directly from the Discovery Engine documentation [here](https://cloud.google.com/generative-ai-app-builder/docs/preview-search-results#genappbuilder\_search-python). You can find a working copy in this notebook [here](https://github.com/GoogleCloudPlatform/generative-ai/blob/main/search/create\_datastore\_and\_search.ipynb).
4. The only change I made was to return the ``response.results`` instead of just the results. Without this, the return type is an object designed to page through results, and that's something we don't need for our basic application.
5. Scroll to the very end of the file and uncomment the entire section below ``Here are the first 5 recipes I found``.
6. Highlight the whole section you just uncommented and open Gemini Code chat. Ask, ``Explain the highlighted code``. If you don't have something selected, Gemini can explain the whole file. If you highlight a section and ask Gemini to explain, or comment, or improve it, Gemini will.

Take a moment and read through the explanation. For what it's worth, using a Colab Enterprise notebook is a great way to explore the Gemini APIs before you integrate them into your application. It's especially helpful at exploring some of the newer APIs which may not be documented as well as they could be.

7. At your editor terminal window, run ``build.sh`` to deploy the final application. Wait until the new version is deployed before moving to the next step.

## ## Task 9: (Optional) Explore the final application

Take a few minutes to explore the final application.

1. In the Google Cloud console, use search to navigate to **Cloud Run**, then click into your **recipe-web-app**.
2. Locate the application test URL (towards the top) and open it in a new browser tab.
3. The application home page should appear. Note the basic layout and navigation provided by Streamlit, with the python files from the ``pages`` folder displayed as navigational choices, and the ``Home.py`` loaded as the home page. Navigate to the **Cooking Advice** page.
4. After a few moments the chat interface will appear. Again, note the nice core layout provided by Streamlit.

5. Try a few cooking related questions and see how the bot functions. Something like:

你有什麼花椰菜的料理建議嗎？

要不要來份經典雞湯食譜？

請介紹蛋白霜。

6. Now let's find a recipe or two. Navigate to the Recipe Search page and try a few searches. Something like:

墨西哥辣肉醬

辣椒、玉米、米飯

檸檬蛋白霜派

含有草莓的甜點

## Congratulations!

You have created an application leveraging Vertex AI Agent Builder applications. Along the way you've explored Gemini Cloud Assist, Gemini Code Assist, and the natural language to SQL features of BigQuery's Data Canvas. Fantastic job!

---